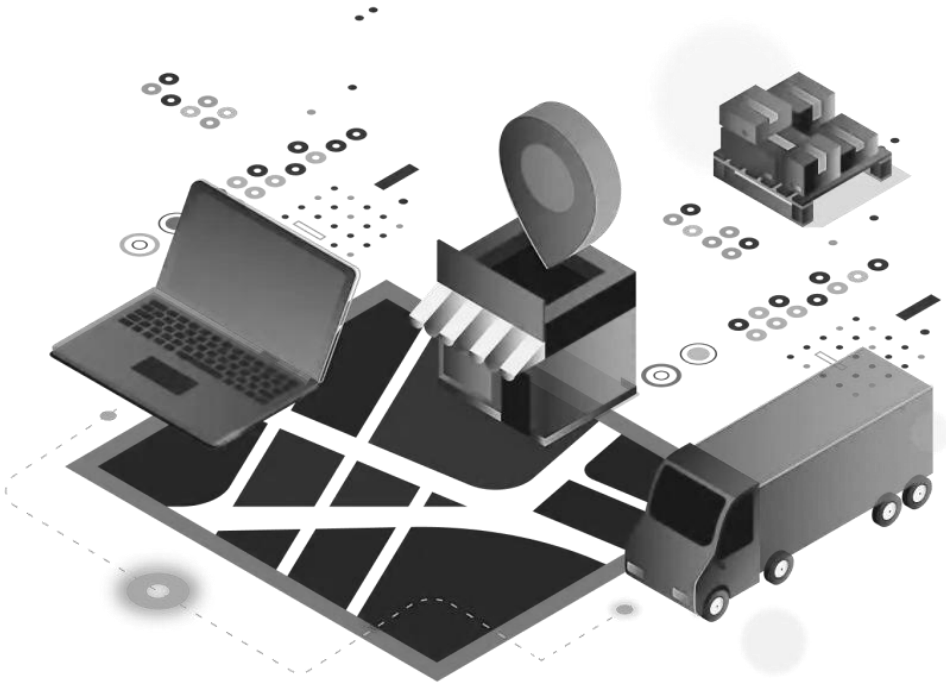
Ingeniería en Sistemas de Información
Ingeniería y Calidad de Software
Año: 2026

Sistema de Gestión de Envíos y Distribución

Trabajo Práctico Integrador - Plan de Pruebas



UTN Facultad Regional San Francisco

Cátedra: Ingeniería y Calidad de Software

Grupo N°7

Docentes:

- Gandino, Diego
- Saldarini, Javier Daniel

Integrantes:

- Araya, Ulises
- Díaz Aimar, Felipe
- Micheloud, Jeremías

Índice

Contenido

Sistema de Gestión de Envíos y Distribución 1
Historial de cambios del documento 3
1. Objetivo de las pruebas 3
2. Alcance (Scope) 3
3. Equipo (People) 5
4. Estrategia (Approach) 5
5. Criterios (Criteria) 7
6. Ambientes (Environments) 8
7. Entregables (Deliverables) 9
8. Gestión de incidentes (Incidents) 10
9. Riesgos (Risks) 12
10. Tareas (Tasks) 13
11. Casos de prueba 14

Historial de cambios del documento

Número de versión	Fecha	Colaborador	Cambios introducidos en esta versión
V1.0	06/10/2026	Araya, U. · Díaz Aimar, F. · Micheloud, J.	Versión inicial del plan. Alcance: HU-01 a HU-05 (Sprint 1, implementadas) y requisitos no funcionales del sistema. Incluye 16 casos de prueba funcionales y 11 no funcionales.

1. Objetivo de las pruebas

Evaluar el producto **Movii** (API REST en Go, panel web del operador/supervisor y app móvil del repartidor) para **verificar** que cada Historia de Usuario cumple sus criterios de aceptación y **validar** que el sistema satisface las expectativas del cliente (la cátedra, en el rol de Product Owner) y de los cuatro perfiles de usuario definidos en el Sprint 0: operador logístico, repartidor, supervisor y cliente final.

Además, las pruebas buscan:

- Reducir la probabilidad de que lleguen a la demo de cada sprint defectos como los encontrados en la revisión de cierre del Sprint 1 (asignaciones no atómicas, pantalla rota con un repartidor sin envíos, validación de token incompleta).
- Medir las características de calidad del producto según **ISO/IEC 25010** que resultan relevantes para un sistema logístico: seguridad, fiabilidad, eficiencia de desempeño, usabilidad, compatibilidad, portabilidad y mantenibilidad.
- Dejar una base de casos de regresión que se reutilice al incorporar las HU de los Sprints 2 y 3.

1.1 Documentos de referencia

Documento	Uso en este plan
Trabajo Integrador — Sprint 0	Propósito del sistema, perfiles de usuario, alcance, stack, DoD, estrategia de QA inicial y decisiones de diseño del prototipo (base de varios RNF).
Sprint Backlog 1	Fichas de HU-01 a HU-05 con sus criterios de aceptación (base de los casos funcionales) y modelo de datos.
Sprint 1 — Implementación y Retrospectiva	Endpoints implementados, defectos encontrados en la revisión de cierre (sección 6.8) y lecciones aprendidas que se incorporan como riesgos y casos de regresión.
ISO/IEC/IEEE 29119-3	Estructura del plan de pruebas (SPACEDIRT).
ISO/IEC 25010	Clasificación de los requisitos no funcionales.
ISTQB CTFL 2018 (sección 5.2.2)	Selección de la estrategia de prueba y técnicas de diseño.

2. Alcance (Scope)

El alcance se define **por funcionalidad** (qué se prueba y qué no) y **por tipo de prueba**. Es un acuerdo entre los integrantes del equipo, en su doble rol de desarrolladores y testers, y con la cátedra como Product Owner.

2.1 Funcionalidades a probar

HU	Funcionalidad	Interfaz / Componente	Estado de desarrollo
HU-01	Registro y autenticación de usuarios, autorización por rol	API + Panel web (login,`/usuarios`) + App móvil (login)	Terminada (Sprint 1)
HU-02	Registro y listado de envíos	API + Panel web (`/envios`)	Terminada (Sprint 1)
HU-03	Gestión de rutas diarias	API + Panel web (`/rutas`)	Terminada (Sprint 1)
HU-04	Asignación y desasignación de envíos a rutas	API + Panel web (detalle de ruta)	Terminada (Sprint 1)
HU-05	Listado de envíos del día para el repartidor	API + App móvil (“Mi ruta de hoy”)	Terminada (Sprint 1)
—	Flujo integral del MVP (operador → repartidor)	API + Web + Móvil	Terminada (Sprint 1)

2.2 Tipos de prueba dentro del alcance

- **Funcionales:** cada criterio de aceptación de HU-01 a HU-05, casos positivos y negativos.
- **No funcionales** (ISO/IEC 25010):
  - **Seguridad:** almacenamiento de contraseñas, autenticación con JWT, autorización por rol, no enumeración de cuentas, inyección SQL, CORS.
  - **Eficiencia de desempeño:** tiempos de respuesta de la API bajo carga y tiempos de carga de pantallas.
  - **Fiabilidad:** concurrencia en la generación del código de guía, tolerancia a fallos de conectividad.
  - **Usabilidad:** diseño mobile-first (390×844), consistencia visual de estados, cantidad de pasos de los flujos críticos, prueba con usuarios y accesibilidad básica.
  - **Compatibilidad:** navegadores para el panel web.
  - **Portabilidad:** app móvil en Android e iOS, instalabilidad del ambiente según el instructivo del Sprint 0.
  - **Mantenibilidad:** cobertura de pruebas unitarias, estándares de codificación y CI.
- **Regresión:** sobre los casos de prioridad Alta en cada nueva versión, y sobre los 8 defectos corregidos en la revisión de cierre del Sprint 1.
- **Aceptación de usuario (UAT):** en la revisión de cada sprint, con la cátedra.

2.3 Fuera del alcance (versión V1.0 del plan)

Elemento excluido	Motivo
HU-06 a HU-10 (Sprint 2) y HU-11 a HU-15 (Sprint 3)	Todavía no implementadas. Se incorporarán al plan en las versiones V1.1 y V1.2, al cerrar cada sprint, siguiendo el mismo formato.
Pantallas de la app móvil que siguen siendo maqueta: confirmar entrega, reportar incidencia, reprogramar, tracking, recuperar contraseña	No están conectadas a la API; corresponden a HU de los Sprints 2 y 3.
Optimización automática de rutas, integración con ERP, facturación, inventario, multi-empresa	Fuera del alcance del producto (Sprint 0, sección 4.2).

Elemento excluido	Motivo
Pruebas de penetración profundas o auditoría de seguridad externa	Exceden la capacidad del equipo. Sí se prueban los controles de seguridad definidos por las HU (ver 11.4).
Pruebas de carga a escala de producción (cientos de usuarios simultáneos), estrés y volumen	No hay un ambiente desplegado (la API corre localmente) y el plan gratuito de Supabase limita las conexiones. Se hace una prueba de carga acotada (ver CPNF-04).
Despliegue en hosting (Render/Railway/Vercel) y documentación Swagger	Pendientes de implementación según el Sprint 0.

3. Equipo (People)

El equipo de pruebas está formado por los tres integrantes del grupo. Como cada uno desarrolla HU propias, la **independencia** se consigue con una **rotación cruzada**: nadie ejecuta los casos de prueba funcionales de una HU que desarrolló. Esa independencia se completa con la cátedra (usuarios de negocio en la UAT) y con usuarios externos al proyecto (compañeros de otros grupos) para las pruebas de usabilidad.

Integrante / Participante	Rol en las pruebas	HU que desarrolló	Pruebas que diseña y ejecuta	Dedicación
Micheloud, Jeremías	Líder de pruebas (planificación, seguimiento, informe de estado). Tester de HU-02 y HU-04.	HU-01	Funcionales de HU-02 y HU-04. No funcionales de seguridad y mantenibilidad.	~2 hs/día
Díaz Aimar, Felipe	Tester de HU-03 y HU-05. Responsable de pruebas móviles.	HU-02, HU-04	Funcionales de HU-03 y HU-05. No funcionales de portabilidad, fiabilidad y desempeño.	~2 hs/día
Araya, Ulises	Tester de HU-01 y del flujo integral. Responsable de pruebas de usabilidad.	HU-03, HU-05	Funcionales de HU-01 y CPF-16 (E2E). No funcionales de usabilidad y compatibilidad.	~2 hs/día
Desarrollador autor de cada HU	Pruebas unitarias y de integración automatizadas (parte del DoD). Corrección de defectos.	—	Suites`go test` y `vitest`	Según HU
Docentes (Gandino, Saldarini)	Product Owner / usuarios de negocio	—	UAT en la revisión de cada sprint	Revisión de sprint
5 usuarios externos (compañeros de otros grupos)	Usuarios representativos de operador y repartidor	—	Prueba de usabilidad (CPNF-09)	1 sesión de ~20 min c/u

4. Estrategia (Approach)

4.1 Tipo de estrategia

Se adopta una estrategia **analítica** (ISTQB CTFL 2018, 5.2.2) que combina:

- **Basada en requisitos:** cada criterio de aceptación de las HU genera al menos un caso de prueba (ver matriz de trazabilidad, 11.2).
- **Basada en riesgos:** la prioridad de cada caso surge del valor de negocio de la HU y de los riesgos de producto (sección 9). Las funciones que, si fallan, frenan la operación (login, alta de envío, asignación) o exponen datos (autorización) tienen prioridad Alta.

Se complementa con una estrategia **reactiva**: sesiones de **prueba exploratoria** de 60 minutos con un objetivo definido (por ejemplo: "romper la asignación de envíos con datos límite") antes de cada revisión de sprint. Esto responde a una lección del Sprint 1: los tests cubrían bien el camino feliz y muy poco los casos vacíos.

4.2 Niveles de prueba y automatización (Pirámide de Cohn)

Nivel	Qué se prueba	Responsable	Herramienta	Automatizada	Cuándo
Unitarias (base)	Lógica de negocio de los servicios de la API (validaciones, generación de código de guía, reglas de asignación) y funciones puras del front (decodificación de JWT, esquemas Zod, adaptadores de datos).	Desarrollador	`testing` + `testify` (Go), `vitest` (web y móvil)	Sí	En cada push, mediante CI
Integración	Repositorios y handlers contra la base PostgreSQL de Supabase real (sin mocks de base de datos).	Desarrollador	`net/http/httptest` (Go)	Sí	En cada push, mediante CI
Sistema — API (E2E HTTP)	Flujo completo por HTTP por bloque de HU (login → operación → verificación).	Desarrollador	Tests en `services/api/tests`	Sí	En cada push, mediante CI
Sistema — UI	Casos funcionales de este plan sobre el panel web y la app móvil, y casos no funcionales.	Tester asignado (rotación)	Navegador, Expo Go, Postman/curl, k6, Lighthouse	No (manual)	Al entrar una HU a testing y en la regresión
Aceptación (UAT) (cima)	Demo de las HU del sprint frente a sus criterios de aceptación, más un recorrido de usabilidad.	Cátedra + equipo	Ambiente de demo	No	Revisión de cada sprint

Criterio de automatización: se automatiza lo que es estable y se repite en cada versión (reglas de negocio, contratos de la API, autorización). Las pruebas de interfaz se hacen manualmente, porque la UI todavía cambia mucho entre sprints y el costo de mantener scripts E2E de UI (por ejemplo, con Playwright o Detox) no se justifica para el tamaño del equipo. Se reevalúa al cerrar el Sprint 2.

4.3 Técnicas de diseño de casos

Técnica	Dónde se aplica
Caja negra — Particiones de equivalencia	Formato de email, roles válidos/inválidos, estados de envío para los filtros.
Caja negra — Análisis de valores límite	Peso del paquete (0 / 0,01 / negativo), longitud mínima de contraseña (7 / 8 caracteres), fecha de la ruta (ayer / hoy en UTC).
Caja negra — Tabla de decisión	Matriz de autorización rol × endpoint (CPF-03).
Caja negra — Transición de estados	Ruta (`planificada` → `en_curso` → `completada`) y envío (`pendiente` ⇄ `asignado`).
Caja blanca — Cobertura de sentencias	Cobertura mínima del 70 % en la lógica de negocio (CPNF-11).
Basada en la experiencia — Exploratoria y predicción de errores	Casos vacíos, lotes mixtos, husos horarios cerca de la medianoche, datos repetidos.

4.4 Regresión

- **Automatizada:** las tres pipelines de CI (`api-ci.yml`, `web-ci.yml`, `mobile-ci.yml`) corren las suites completas en cada push. Los tests de la API se ejecutan en serie (`go test ./... -p 1`), para que dos paquetes no toquen la base compartida al mismo tiempo.

- **Manual:** cada vez que una nueva versión llega a testing se re-ejecutan todos los casos de prioridad **Alta**. Los 8 defectos corregidos en la revisión del Sprint 1 tienen un caso de regresión explícito (CPF-02, CPF-07, CPF-12, CPF-14, CPF-15, CPNF-01 y CPNF-11).

5. Criterios (Criteria)

5.1 Criterios de entrada (Definition of Ready para testing)

Una HU entra en testing de sistema cuando:

1. El desarrollador la marcó como terminada en Jira y el pull request fue revisado y aprobado por otro integrante.
2. Las pipelines de CI de los componentes afectados están en verde (unitarias + integración + E2E HTTP).
3. La API levanta sin errores contra la base de pruebas, con las migraciones aplicadas (`go run ./cmd/migrate up`) y los datos semilla cargados (usuarios demo de los tres roles).
4. El ambiente de prueba (sección 6) está disponible: navegador, teléfono con Expo Go en la misma red y acceso al editor SQL de Supabase.
5. Los casos de prueba de la HU fueron revisados por un segundo integrante (paso 9 de "Cómo construir un caso de prueba").

5.2 Criterios de salida (Definition of Done de las pruebas)

Las pruebas de un sprint se consideran completas cuando se ejecutaron **todos** los casos planificados o, si el tiempo apremia, al menos:

- El **100 %** de los casos de prioridad **Alta** se ejecutaron **sin fallos**.
- Al menos el **80 %** de los casos de prioridad **Media** se ejecutaron sin fallos, y los que fallan tienen un defecto registrado de severidad Media o Baja.
- Los casos de prioridad **Baja** se ejecutan si queda capacidad.

Prioridad de los casos de prueba	¿Qué significa?
Alta	El caso**debe** ser ejecutado
Media	El caso**debería** ser ejecutado
Baja	El caso**puede** ser ejecutado

5.3 Criterio para aprobar la entrega (pasar a la demo / "producción")

- No hay defectos **Bloqueantes** ni **Críticos** abiertos. Esto es consistente con el punto 8 del DoD del equipo.
- Los defectos de severidad **Alta** abiertos tienen una corrección planificada y fueron aceptados explícitamente por el Product Owner.
- Las tres pipelines de CI están en verde sobre la rama que se presenta.

5.4 Criterios de suspensión y reanudación

Se suspende la ejecución cuando…	Se reanuda cuando…
Falla el login (CPF-02) o la validación de token: todo el resto del sistema depende de HU-01.	Se corrige el defecto y CPF-02 y CPF-03 pasan.
La base de Supabase no está disponible o los datos de prueba quedaron corruptos (por ejemplo, por dos corridas de tests en paralelo).	Se restaura la base (migración down/up + seed) y se verifica con CPF-02 y CPF-07.
Más del 30 % de los casos de una HU falla en la primera ejecución.	El desarrollador entrega una nueva versión con la CI en verde. La HU vuelve a “En desarrollo” mientras tanto.
La build de la app móvil no abre en Expo Go (incompatibilidad de SDK, como la del Sprint 1).	La app abre en los dos dispositivos de prueba.

6. Ambientes (Environments)

6.1 Hardware

Equipo	Uso
PC/notebooks del equipo con Windows 11	Ejecución de la API en Go (`:8080`), del panel web (Vite, `:5173`), de Metro/Expo (`:8081`) y de las herramientas de prueba.
1 iPhone físico (iOS) con Expo Go	Pruebas de HU-05, portabilidad y usabilidad móvil.
1 teléfono Android físico con Expo Go	Pruebas de HU-05, portabilidad y usabilidad móvil.
Red WiFi local compartida	Los teléfonos acceden a la API por la IP local de la PC (`EXPO_PUBLIC_API_URL`).

6.2 Software de base

Componente	Versión
Go	1.22 o superior
Node.js	20 LTS (local) / 22 (CI)
pnpm	9
Expo SDK	57 (Expo Go compatible en ambos teléfonos)
Base de datos	PostgreSQL gestionado por Supabase
Navegadores (panel web)	Google Chrome, Mozilla Firefox y Microsoft Edge, en su última versión estable al momento de la ejecución. La versión exacta se registra en cada ejecución.

6.3 Herramientas de prueba

Herramienta	Uso
`go test` + `testify`, `vitest`	Pruebas unitarias e integración automatizadas
GitHub Actions	Integración continua (lint, tipos, tests)
Postman / `curl`	Pruebas manuales de la API (casos negativos que la UI no permite generar)
k6	Prueba de carga y de concurrencia acotada (CPNF-04)

Herramienta	Uso
Chrome DevTools (modo dispositivo 390×844) y Lighthouse	Usabilidad, accesibilidad y tiempos de carga del panel web
Editor SQL de Supabase	Preparar precondiciones que todavía no tienen pantalla (pasar una ruta a `en_curso`, un envío a `entregado`, un usuario a inactivo) y verificar la persistencia
Script de firma de JWT (Go o Node, con el `JWT_SECRET` de pruebas)	Generar tokens expirados o con otro algoritmo (CPNF-01)
GitHub Issues	Registro y seguimiento de defectos
Jira	Seguimiento de las tareas de prueba del sprint

6.4 Datos de prueba

- **Usuarios semilla** (migración `0002_seed`): `supervisor@movii.demo`, `operador@movii.demo` y `repartidor@movii.demo`, todos con la contraseña de demo `Movii2026!`.
- **Usuarios adicionales creados por los casos:** un segundo repartidor (`repartidor2@movii.test`) para probar el aislamiento de datos, y un usuario inactivo (`inactivo@movii.test`).
- Los datos creados durante las pruebas usan el sufijo `@movii.test` o el prefijo `TEST-` en nombres de rutas y destinatarios, para poder limpiarlos con una consulta SQL al final de cada ciclo.
- No se usan datos personales reales.
- **Base de datos:** se usa un proyecto de Supabase dedicado a pruebas, separado del que se usa para la demo, para evitar que una corrida de pruebas deje datos inconsistentes en la demo (problema ocurrido en el Sprint 1).
- **Fechas:** todo el sistema trabaja en **UTC**. Los casos que dependen de "hoy" se ejecutan evitando la franja de las 21:00 a 00:00 hora argentina (UTC−3), salvo el caso que prueba ese borde a propósito.

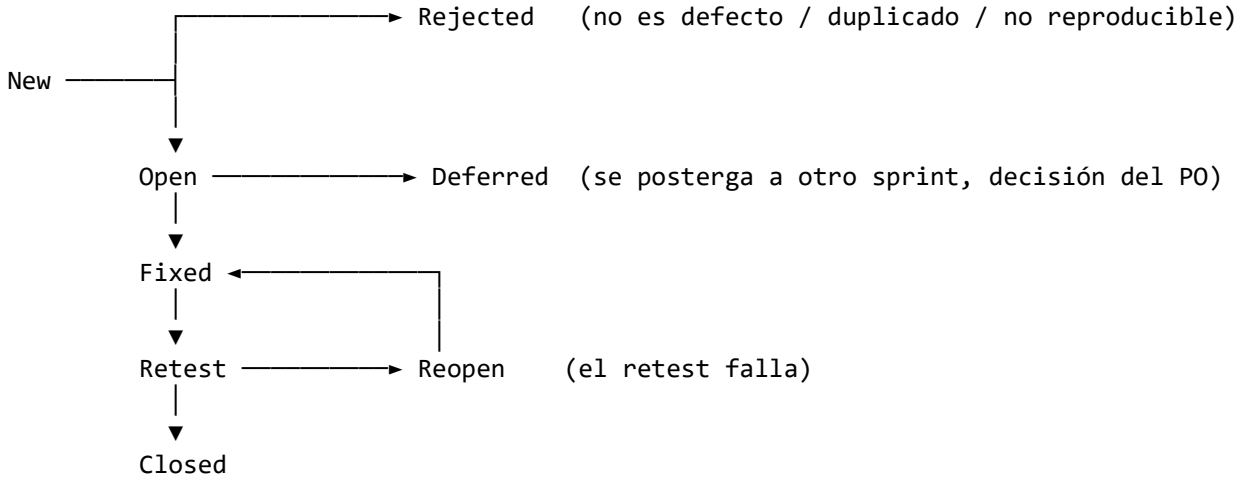
7. Entregables (Deliverables)

Entregable	Contenido	Cuándo
Plan de pruebas (este documento)	Planificación SPACEDIRT, casos de prueba y trazabilidad	V1.0 ahora. Se actualiza al cierre de cada sprint.
Matriz de trazabilidad	Requisito (CA / RNF) → caso de prueba → resultado	Con cada versión del plan
Registro de ejecución de casos	Por cada caso ejecutado: fecha, versión (commit), ejecutado por, ambiente (navegador/dispositivo), estado de cada paso, resultado obtenido, defecto asociado	En cada ciclo de ejecución
Evidencias	Capturas de pantalla, salidas de `curl`, reportes de k6 y Lighthouse, logs de CI	Adjuntas al registro de ejecución
Reporte de defectos	Issues en GitHub con la plantilla de la sección 8.3	Durante la ejecución
Informe de estado del sprint	Casos ejecutados, aprobados y fallidos por prioridad; defectos por severidad y estado; cumplimiento de los criterios de salida; riesgos materializados	En la revisión de cada sprint
Checklist de regresión	Lista de los casos de prioridad Alta a re-ejecutar en cada versión	Con cada versión del plan

8. Gestión de incidentes (Incidents)

8.1 Flujo de estados del defecto

Se utiliza el siguiente flujo, implementado en GitHub Issues con una etiqueta por estado y un tablero (GitHub Projects) con una columna por estado:



Estado	Quién lo mueve	Significado
New	Tester	Defecto reportado, pendiente de análisis
Open	Líder de pruebas	Confirmado y asignado a un desarrollador
Rejected	Líder de pruebas + desarrollador	No es un defecto, está duplicado o no se pudo reproducir (se documenta el motivo)
Deferred	Product Owner	Se corrige en un sprint posterior. Solo se admite en severidad Media o Baja.
Fixed	Desarrollador	Corregido. El PR incluye un test de regresión automatizado y referencia el issue.
Retest	Tester	En re-ejecución del caso que lo detectó
Reopen	Tester	El retest falló; vuelve al desarrollador
Closed	Tester	El retest pasó y la CI está en verde

Responsabilidades: la **severidad** la asigna el tester que reporta. La **prioridad** de corrección la asigna el Product Owner; durante el sprint, el equipo la propone y la valida con la cátedra en la revisión.

8.2 Escala de severidad

Severidad	Criterio	Ejemplo en Movii
Bloqueante	Una funcionalidad no se puede usar y no hay otra forma de realizar la acción.	No se puede iniciar sesión; no se puede registrar ningún envío.
Crítica	Una funcionalidad no se puede usar, pero existe un camino alternativo; o hay datos inconsistentes o expuestos.	La asignación responde error pero deja envíos asignados igual; un repartidor ve envíos de otro.
Alta	Defecto sobre una funcionalidad prioritaria, o con impacto secundario en una funcionalidad prioritaria.	El resumen del día muestra contadores incorrectos.

Severidad	Criterio	Ejemplo en Movii
Media	Defecto que impacta en una funcionalidad secundaria.	El filtro por fecha del listado de envíos no combina con el filtro por estado.
Baja	Defecto cosmético o error de ortografía.	Un badge de estado con un color que no corresponde; un texto mal escrito.

Los defectos **Bloqueantes** y **Críticos** bloquean el cierre del sprint (DoD, punto 8).

8.3 Plantilla de reporte de defecto (issue de GitHub, etiqueta `bug`)

Campo	Descripción
ID	`DEF-<n>` (número del issue)
Título	Resumen corto del fallo observado
Módulo / HU	Ej.: HU-04 — Asignación
Caso de prueba	Ej.: CPF-12 (c)
Versión / commit	Hash del commit probado
Ambiente	Navegador y versión, o dispositivo y SO
Precondiciones	Estado de datos necesario
Pasos para reproducir	Numerados
Resultado esperado	Según el caso de prueba
Resultado obtenido	Lo que pasó, con su evidencia
Adjuntos	Capturas, salida de`curl`, logs
Severidad	Bloqueante / Crítica / Alta / Media / Baja
Prioridad	La asigna el PO
Detectado por / Asignado a	Tester / Desarrollador
Estado	Según el flujo de 8.1

Ejemplo, tomado de un defecto real de la revisión del Sprint 1 (hoy en estado Closed):

Campo	Valor
ID	DEF-02
Título	La asignación de un lote mixto responde 409 pero deja asignado el envío pendiente
Módulo / HU	HU-04 — Asignación de envíos
Caso de prueba	CPF-12, variante (c)
Pasos	(1) Tener el envío A en estado`pendiente` y el envío B en estado `asignado`. (2) `POST /rutas/{id}/envios` con `{"envio_ids":["A","B"]}`. (3) Consultar el envío A.
Esperado	409`envio_no_disponible`; ningún envío del lote cambia de estado.
Obtenido	409, pero A quedó en`asignado` y vinculado a la ruta.
Severidad	Crítica (datos inconsistentes entre lo que se informa y lo que se persiste)
Corrección	El`UPDATE` corre dentro de una transacción con rollback, se descartan los ids repetidos y se agregó un test de regresión.
Estado	Closed

9. Riesgos (Risks)

Los riesgos se describen como **causa → consecuencia** ("Dado que… posiblemente…"). Probabilidad e impacto: Baja / Media / Alta.

9.1 Riesgos del proyecto (de las pruebas)

#	Riesgo	Probabilidad	Impacto	Mitigación (prevenir)	Contingencia (si sucede)
RP-1	Dado que el equipo tiene ~2 hs/día por integrante y en el Sprint 1 se excedió un 14 % lo estimado, posiblemente no se ejecuten todos los casos antes de la revisión.	Alta	Alto	Estimar las tareas de prueba en el sprint backlog. Seguimiento semanal del burndown. Ejecutar primero los casos de prioridad Alta.	Aplicar el criterio de salida reducido (100 % Alta, 80 % Media) y diferir los de prioridad Baja. Informarlo en el informe de estado.
RP-2	Dado que los mismos integrantes desarrollan y prueban, posiblemente haya sesgo de confirmación y se den por cumplidos criterios sin verificarlos (pasó con HU-05 en el Sprint 1).	Media	Alto	Rotación cruzada de testers (sección 3). Ejecución CA por CA según la matriz de trazabilidad. Revisión de casos por un segundo integrante.	Re-ejecución completa de la HU por otro integrante antes de la demo.
RP-3	Dado que la base de Supabase es compartida, posiblemente una corrida de tests en paralelo o un caso mal limpiado corrompa los datos de prueba.	Media	Medio	Proyecto de Supabase dedicado a pruebas.`go test -p 1`. Datos con prefijo `TEST-`.	Restaurar la base con migración down/up + seed (procedimiento documentado, ~10 min).
RP-4	Dado que se usa el plan gratuito de Supabase, posiblemente el proyecto se pause por inactividad o se alcance el límite de conexiones durante la prueba de carga.	Media	Medio	Reactivar el proyecto antes de cada ciclo. Limitar k6 a 20 usuarios virtuales.	Reprogramar la prueba de carga. Usar un Postgres local solo para CPNF-04.
RP-5	Dado que Expo Go depende de la versión del SDK, posiblemente una actualización de Expo Go deje la app sin abrir en un teléfono (pasó con SDK 54 → 57).	Media	Alto	No actualizar Expo Go en los teléfonos de prueba durante el sprint. Fijar las versiones de las dependencias.	Probar en el teléfono que sí funciona y en el modo web/emulador. Registrar la limitación.
RP-6	Dado que la ejecución de pruebas manuales depende de que los tres integrantes coincidan en la misma red local, posiblemente se demore la ejecución de los casos móviles.	Media	Bajo	Agendar dos sesiones fijas por semana para pruebas móviles.	Usar el túnel de Expo (`--tunnel`) para probar desde otra red.

9.2 Riesgos del producto (qué puede fallar en el software)

#	Riesgo	Probabilidad	Impacto	Cómo lo cubren las pruebas

#	Riesgo	Probabilidad	Impacto	Cómo lo cubren las pruebas
RPr-1	Dado que la autorización se valida por rol en cada endpoint, posiblemente un rol acceda a datos o acciones que no le corresponden (por ejemplo, un repartidor lista los envíos de todos los clientes).	Media	Alto	CPF-03, CPNF-02
RPr-2	Dado que la asignación modifica varios envíos a la vez, posiblemente quede un estado inconsistente si una parte falla.	Media	Alto	CPF-12, CPF-13
RPr-3	Dado que el sistema calcula “hoy” en UTC, posiblemente cerca de la medianoche se rechacen rutas válidas o se fechen mal los códigos de guía.	Media	Medio	CPF-05, CPF-12, CPF-14
RPr-4	Dado que el código de guía se genera contando los envíos del día, posiblemente dos altas simultáneas generen el mismo código.	Baja	Alto	CPF-05, CPNF-04
RPr-5	Dado que el repartidor arranca el día muchas veces sin envíos asignados, posiblemente la app muestre un error en vez de un estado vacío.	Media	Alto	CPF-14
RPr-6	Dado que el login es la puerta de entrada, posiblemente las respuestas o los tiempos de respuesta permitan saber qué emails están registrados.	Baja	Medio	CPF-02, CPNF-01

10. Tareas (Tasks)

Planificación de las tareas de prueba dentro de cada sprint de 4 semanas (se repite en los Sprints 2 y 3, agregando los casos de las HU nuevas). ■ = semana en la que se realiza la tarea.

Tarea	Responsable	Sem. 1	Sem. 2	Sem. 3	Sem. 4	Est. (hs)
Planificación y actualización del plan de pruebas	Micheloud	■				4
Diseño / actualización de casos de prueba de las HU del sprint	Los tres (rotación)	■	■			6
Revisión cruzada de los casos de prueba	Los tres		■			2
Preparación del ambiente y de los datos de prueba	Díaz Aimar	■		■		3
Pruebas unitarias e integración (DoD de cada HU)	Desarrollador de cada HU	■	■	■	■	(en cada HU)
Ejecución de pruebas funcionales (API + web + móvil)	Tester asignado		■	■	■	6
Ejecución de pruebas no funcionales de seguridad y mantenibilidad	Micheloud			■		3
Diseño y ejecución de la prueba de carga (k6)	Díaz Aimar			■		3
Ejecución de pruebas de portabilidad y compatibilidad	Díaz Aimar / Araya			■	■	3
Diseño y ejecución de la prueba de usabilidad con usuarios	Araya		■		■	4

Tarea	Responsable	Sem. 1	Sem. 2	Sem. 3	Sem. 4	Est. (hs)
Sesiones de prueba exploratoria (60 min)	Los tres			■	■	3
Reporte de defectos	Quien detecta		■	■	■	(continuo)
Re-test y regresión de casos de prioridad Alta	Tester asignado			■	■	4
Seguimiento y control (burndown, métricas de defectos)	Micheloud	■	■	■	■	2
Confección del informe de estado del sprint	Micheloud				■	2
UAT en la revisión de sprint	Equipo + cátedra				■	1
Total estimado por sprint						**~46**

11. Casos de prueba

11.1 Formato del caso de prueba

Cada caso sigue el paso a paso de la plantilla "Cómo construir un caso de prueba" de la cátedra y los campos vistos en clase (IEEE 610: *"un conjunto de entradas de prueba, condiciones de ejecución y resultados esperados desarrollados con un objetivo particular"*).

Para que el conjunto sea manejable, cada caso agrupa **un escenario completo de una HU**: el camino válido y sus variantes inválidas o de borde se incluyen como **datos de prueba** y **pasos** del mismo caso, identificados con letras (a), (b), (c)… El caso se aprueba solo si todas sus variantes dan el resultado esperado. Si una variante falla, el defecto se registra indicando el caso y la variante (por ejemplo, "CPF-01 (c)").

Campo	Contenido
ID / Título	`CPF-nn` (funcional) o `CPNF-nn` (no funcional) y un título descriptivo
Objetivo	Qué funcionalidad o característica se verifica (paso 1)
Requisito asociado	HU y criterios de aceptación, o característica ISO/IEC 25010 (paso 2)
Clasificación	Tipo (funcional o no funcional) · nivel (API / Web / Móvil / E2E) · técnica de diseño · prioridad Alta / Media / Baja (pasos 1 y 10)
Precondiciones	Configuración, datos iniciales y estado del sistema (paso 3)
Datos de prueba	Entradas concretas de cada variante (paso 5)
Pasos	Acciones numeradas (paso 4)
Resultado esperado	Claro y medible, para cada variante (paso 6)
Resultado alternativo	Comportamiento esperado ante escenarios alternativos, si aplica (paso 7)
Postcondición	Estado en el que queda el sistema (se omite cuando el caso no modifica datos)
Responsables	Integrante que lo diseña, integrante que lo ejecuta y estado: Pendiente / En ejecución / Aprobado / Fallido / Bloqueado (paso 10). Todos los casos de V1.0 están en estado**Pendiente**.

Los campos de **ejecución** (fecha, versión del build, ambiente, resultado obtenido por paso, defecto asociado, ejecutado por) se completan en el Registro de ejecución (entregable de la sección 7).

Convenciones para los casos de API: `{{API}}` = `http://<ip-local>:8080`. `TOKEN_SUP`, `TOKEN_OPE` y `TOKEN_REP` son los tokens obtenidos con `POST /auth/login` para el supervisor, el operador y el repartidor semilla. Todas las peticiones con cuerpo llevan `Content-Type: application/json`.

11.2 Matriz de trazabilidad (Requisito → Caso de prueba)

Cada criterio de aceptación (su descripción completa está en el Sprint Backlog 1) se indica con el caso que lo verifica.

HU	Criterio de aceptación → Caso de prueba
HU-01	CA-01 → CPF-01, CPNF-01 · CA-02 → CPF-01 · CA-03 → CPF-02 · CA-04 → CPF-02, CPNF-01 · CA-05 → CPF-03, CPNF-01, CPNF-02 · CA-06 → CPF-04
HU-02	CA-01 → CPF-05 · CA-02 → CPF-05, CPNF-04 · CA-03 → CPF-05 · CA-04 → CPF-06 · CA-05 → CPF-07 · CA-06 → CPF-06, CPF-03
HU-03	CA-01 → CPF-08 · CA-02 → CPF-09 · CA-03 → CPF-10 · CA-04 → CPF-09 · CA-05 → CPF-08
HU-04	CA-01 → CPF-11, CPF-12 · CA-02 → CPF-11, CPF-12 · CA-03 → CPF-11 · CA-04 → CPF-13 · CA-05 → CPF-12 · CA-06 → CPF-11, CPF-13
HU-05	CA-01 → CPF-14 · CA-02 → CPF-14 · CA-03 → CPF-14 · CA-04 → CPF-15 · CA-05 → CPF-15 · CA-06 → CPNF-08
MVP	Objetivo del Sprint 1 (flujo operador → repartidor) → CPF-16

Característica ISO/IEC 25010	Fuente del requisito	Casos de prueba
Seguridad — Confidencialidad / Autenticidad	HU-01 CA-01, CA-03, CA-04, CA-05	CPNF-01
Seguridad — Confidencialidad (autorización)	HU-01 (“PARA” y CA-05); HU-05 CA-01	CPNF-02
Seguridad — Integridad	Stack (SQL explícito con pgx);`CORS_ORIGINS` (Sprint 0, 7.2)	CPNF-03
Eficiencia de desempeño / Fiabilidad — Madurez	Valor de negocio “eficiencia operativa” (Sprint 0, 3.1); HU-02 CA-02	CPNF-04, CPNF-05
Compatibilidad / Portabilidad — Adaptabilidad	Una API para dos interfaces; Android e iOS desde un mismo proyecto (Sprint 0, 6.1 y 6.2)	CPNF-06, CPF-16
Portabilidad — Instalabilidad	Instructivo de instalación (Sprint 0, 7.2)	CPNF-07
Usabilidad — Operabilidad / Estética / Accesibilidad	HU-05 CA-06; decisiones de diseño del prototipo (Sprint 0, 8.4)	CPNF-08, CPNF-09
Fiabilidad — Tolerancia a fallos	Operación del repartidor con conectividad variable	CPNF-10
Mantenibilidad — Capacidad de prueba / Modificabilidad	DoD puntos 2, 4 y 5	CPNF-11

11.3 Casos de prueba funcionales (RF)

HU-01 — Registro y autenticación de usuarios

CPF-01 — Alta de usuarios y validación de sus datos

Objetivo	Verificar que el supervisor puede registrar un usuario con nombre, email, contraseña y rol; que se persiste con la contraseña hasheada; y que se rechazan los datos inválidos con un mensaje claro.

Requisito	HU-01 · CA-01, CA-02 (y política de contraseñas, tarea T-01.03)
Clasificación	Funcional · Web + API + BD · Particiones de equivalencia y valores límite · Prioridad**Alta**
Precondiciones	Sesión iniciada en el panel web como`supervisor@movii.demo`. `TOKEN_SUP` vigente. Los emails de prueba no existen en la tabla `usuarios`.
Datos de prueba	(a)**Válido:** `TEST Operador Uno` · `operador1@movii.test` · `Prueba2026!` · rol `operador`. (b) **Email duplicado:** `operador1@movii.test`, y también `OPERADOR1@movii.test`. (c) **Email con formato inválido:** `sinarroba.movii.test`, `usuario@`, `""`. (d) **Rol inválido:** `admin`, `Operador`. (e) **Contraseña en el límite:** `Abc1234` (7 caracteres) y `Abc12345` (8 caracteres).
Pasos	(1) En**Usuarios → Nuevo usuario**, crear el usuario (a). (2) Verificar que aparece en el listado y ejecutar `SELECT rol, activo, password_hash FROM usuarios WHERE email='operador1@movii.test';`. (3) Intentar crear (b), (c) y (e) desde el formulario. (4) Enviar (c), (d) y la contraseña de 7 caracteres de (e) directamente a `POST {{API}}/auth/register`.
Resultado esperado	(a) 201. El usuario aparece en el listado, activo y con rol`operador`; en la base, `password_hash` empieza con `$2a$10$` y es distinto de la contraseña. (b) 409 “ya existe un usuario con ese email”, también en la variante en mayúsculas. (c) 400 “el email no tiene un formato válido”. (d) 400 “el rol debe ser operador, repartidor o supervisor”. (e) Con 7 caracteres, la web muestra “La contraseña debe tener al menos 8 caracteres” y la API también rechaza con 400; con 8 caracteres el usuario se crea. Ninguna variante inválida crea registros.
Resultado alternativo	Si la API acepta la contraseña de 7 caracteres, se registra un defecto de severidad Alta: la política se valida solo en el front y se puede saltear. Si la variante en mayúsculas de (b) se acepta, se registra un defecto de severidad Media.
Postcondición	Existen`operador1@movii.test` y el usuario de 8 caracteres de (e).
Responsables	Diseña: Araya, U. · Ejecuta: Araya, U. · Estado: Pendiente

CPF-02 — Inicio de sesión con credenciales válidas e inválidas

Objetivo	Verificar que un usuario registrado y activo recibe un JWT con su rol, y que cualquier intento inválido recibe el mismo error 401, sin revelar si el email existe.
Requisito	HU-01 · CA-03, CA-04 (regresión del defecto#8 del Sprint 1: usuario inactivo)
Clasificación	Funcional · API + Web + Móvil · Particiones de equivalencia · Prioridad**Alta**
Precondiciones	Usuarios semilla cargados. Usuario`inactivo@movii.test` / `Prueba2026!` creado y desactivado con `UPDATE usuarios SET activo=false WHERE email='inactivo@movii.test';`.
Datos de prueba	(a)**Válido:** `operador@movii.demo` / `Movii2026!`. (b) Contraseña incorrecta: `operador@movii.demo` / `Incorrecta1`. (c) Email inexistente: `noexiste@movii.test` / `Movii2026!`. (d) Usuario inactivo con su contraseña correcta.
Pasos	(1) Enviar`POST {{API}}/auth/login` con (a) y decodificar el payload del token. (2) Iniciar sesión con (a) en el panel web. (3) Enviar (b), (c) y (d) a la API y comparar las respuestas. (4) Repetir (b) y (c) en el login de la web y en el de la app móvil.
Resultado esperado	(a) 200 con`{"token":"<jwt>"}`. El header del token indica `HS256` y el payload contiene `sub`, `rol: "operador"` y `exp` ≈ ahora + 24 h. La web redirige a `/envios`. (b), (c) y (d) responden 401 con un cuerpo idéntico (`credenciales_invalidas`, “email o contraseña incorrectos”) y sin token. La web y la app muestran el mismo mensaje para (b) y (c).
Postcondición	Sesión iniciada con el usuario (a).
Responsables	Diseña: Araya, U. · Ejecuta: Araya, U. · Estado: Pendiente

CPF-03 — Control de acceso por token y por rol

Objetivo	Verificar que los endpoints protegidos rechazan las peticiones sin un token válido, que cada rol solo ejecuta las operaciones que le corresponden y que la web solo le muestra a cada perfil sus secciones.
Requisito	HU-01 · CA-05 · HU-02 · CA-06 · HU-05 · CA-01
Clasificación	Funcional · API + Web · Tabla de decisión · Prioridad**Alta**
Precondiciones	`TOKEN_SUP`, `TOKEN_OPE` y `TOKEN_REP` vigentes. Existen una ruta planificada para hoy (`RUTA_ID`) y un envío pendiente (`ENVIO_ID`).
Datos de prueba	(a)**Sin token o con token mal formado:** sin header `Authorization`; `Bearer` vacío; `Bearer abc.def.ghi`. (b) **Tabla de decisión rol × endpoint** (código esperado en cada celda):

Endpoint	Supervisor	Operador	Repartidor
`GET /users`	200	403	403
`POST /envios`	201	201	403
`POST /rutas` y `PUT /rutas/{id}`	201 / 200	201 / 200	403
`POST /rutas/{id}/envios` y `DELETE /rutas/{id}/envios/{envioId}`	200	200	403
`GET /repartidor/envios-del-dia`	403	403	200

Pasos	(1) Enviar`GET {{API}}/envios` con cada variante de (a). (2) Para cada celda de (b), enviar la petición con el token del rol correspondiente y un cuerpo válido; registrar el código obtenido. (3) En la web, sin sesión, navegar a `/envios`. (4) Iniciar sesión en la web como repartidor, luego como operador (navegar a `/usuarios`) y luego como supervisor; revisar el menú en cada caso.
Resultado esperado	(1) 401 (`sin_token` o `token_invalido`) y ningún dato. (2) Cada celda coincide con lo esperado; los 403 devuelven `rol_no_autorizado` (“no tenés permisos para esta acción”) y no modifican datos. (3) Redirige a `/login`. (4) El repartidor ve la pantalla “Acceso no disponible” (su rol usa la app móvil). El operador ve Envíos y Rutas, no ve Usuarios, y al navegar a `/usuarios` se le niega el acceso. El supervisor ve Envíos, Rutas y Usuarios.
Postcondición	Los registros creados por los 201 se limpian al final del ciclo (prefijo`TEST-`).
Responsables	Diseña: Araya, U. · Ejecuta: Araya, U. · Estado: Pendiente

CPF-04 — Listado de usuarios para el supervisor

Objetivo	Verificar que el supervisor ve la lista de usuarios con nombre, email, rol y estado activo.
Requisito	HU-01 · CA-06
Clasificación	Funcional · Web + API · Prioridad**Media**
Precondiciones	Sesión de supervisor. Existen los 3 usuarios semilla y`inactivo@movii.test`.
Pasos	(1) En el panel web, ir a**Usuarios**. (2) Revisar las columnas y los registros. (3) Enviar `GET {{API}}/users` con `TOKEN_SUP` y revisar el cuerpo.
Resultado esperado	Se listan todos los usuarios con nombre, email, rol y estado;`inactivo@movii.test` figura como inactivo. La respuesta de la API **no** incluye `password_hash`. (El rechazo para los demás roles se verifica en CPF-03.)
Responsables	Diseña: Araya, U. · Ejecuta: Araya, U. · Estado: Pendiente

HU-02 — Registro de envíos

CPF-05 — Registro de un envío y generación del código de guía

Objetivo	Verificar que el operador registra un envío con todos los datos, que se genera un código de guía único con el formato definido, y que el envío queda "pendiente" con su fecha de creación.
Requisito	HU-02 · CA-01, CA-02, CA-03
Clasificación	Funcional · Web + API + BD · Partición de equivalencia (clase válida) · Prioridad**Alta**
Precondiciones	Sesión iniciada como`operador@movii.demo`. `TOKEN_OPE` vigente.
Datos de prueba	(a)**Formulario completo:** remitente `Depósito Central`, `3564-400000`, `Bv. 25 de Mayo 1500`; destinatario `TEST Cliente Uno`, `3564-411111`, `Mitre 250`, `2B`, `San Francisco`, `X2400`; paquete 3,5 kg, `mediano`, `Ropa`, frágil ✓, urgente ✗. (b) **Cuerpo mínimo por API**, enviado 3 veces seguidas: `{"remitente":{"nombre":"TEST Rem"},"destinatario":{"nombre":"TEST Dest","calle":"Iturraspe","numero":"100","localidad":"San Francisco","codigo_postal":"X2400"},"paquete":{"peso":1,"tamano":"chico"}}`
Pasos	(1) Ir a**Envíos → Nuevo envío** y verificar que el formulario muestra todos los campos de remitente, destinatario y paquete (tamaño chico/mediano/grande, frágil, urgente). (2) Registrar (a) y anotar el código de guía. (3) Abrir el detalle del envío y verificar las tablas `envios`, `remitentes` y `destinatarios`. (4) Enviar (b) tres veces a `POST {{API}}/envios`. (5) Ejecutar `SELECT codigo_guia, count(*) FROM envios GROUP BY codigo_guia HAVING count(*)>1;`.
Resultado esperado	(1) Se muestran todos los campos de CA-01. (2) El código cumple`^MOV-\d{8}-\d{4}$`, con la fecha UTC del día. (3) Estado `pendiente`, fecha de creación del día y datos idénticos a los ingresados; en la base, un remitente, un destinatario y un envío vinculados, con `fragil=true`, `urgente=false` y `ruta_id` nulo. (4) Tres 201 con códigos consecutivos del mismo día. (5) La consulta no devuelve filas (no hay duplicados).
Postcondición	4 envíos pendientes:`ENV-A` (el de la web) y `ENV-B`, `ENV-C`, `ENV-D`. Se usan en HU-04.
Responsables	Diseña: Micheloud, J. · Ejecuta: Micheloud, J. · Estado: Pendiente

CPF-06 — Validación de datos obligatorios y del peso

Objetivo	Verificar que no se puede registrar un envío sin los datos obligatorios o con un peso inválido, que el formulario indica qué campo falta, y que la API valida lo mismo aunque la petición no venga de la web.
Requisito	HU-02 · CA-04, CA-06
Clasificación	Funcional · Web + API · Particiones de equivalencia y valores límite · Prioridad**Alta**
Precondiciones	Sesión de operador en**Envíos → Nuevo envío**. `TOKEN_OPE` y `TOKEN_REP` vigentes.
Datos de prueba	(a)**Un obligatorio vacío por vez**, partiendo de los datos de CPF-05 (a): nombre del destinatario, calle, localidad, peso. (b) **Todos vacíos.** (c) **Peso en los límites:** `0`, `-1`, `0.01`, `9999.99`. (d) **API sin pasar por el formulario:** destinatario sin nombre; cuerpo que no es JSON (`hola`). (e) **Rol no autorizado:** el cuerpo válido de CPF-05 (b) enviado con `TOKEN_REP`.
Pasos	(1) En la web, intentar registrar cada variante de (a), (b) y (c), y verificar en la pestaña Network si se envía el`POST`. (2) Enviar (c) y (d) a `POST {{API}}/envios`. (3) Enviar (e). (4) Contar los envíos antes y después.
Resultado esperado	(a) Mensaje específico en cada campo: "El nombre del destinatario es obligatorio", "La calle es obligatoria", "La localidad es obligatoria", "El peso debe ser mayor a 0"; no se envía la petición. (b) Se muestran los cuatro mensajes juntos. (c)`0` y `-1` se rechazan (en la web con el mensaje del peso; en la API con 400 `campos_obligatorios`); `0.01` y `9999.99` se aceptan. (d) 400 `campos_obligatorios` y 400 `body_invalido`, sin dejar datos parciales en la base. (e) 403 `rol_no_autorizado`. Solo cambia la cantidad de envíos por las dos altas válidas de (c).

Postcondición	2 envíos nuevos (pesos 0,01 y 9999,99).
Responsables	Diseña: Micheloud, J. · Ejecuta: Micheloud, J. · Estado: Pendiente

CPF-07 — Listado de envíos con filtros y consulta de detalle

Objetivo	Verificar que el listado muestra las columnas requeridas, que los filtros por estado y fecha funcionan solos y combinados, y que el detalle de un envío inexistente se maneja sin errores.
Requisito	HU-02 · CA-05 (regresión del defecto#6 del Sprint 1: fecha de creación)
Clasificación	Funcional · Web + API · Particiones de equivalencia (una clase por estado) · Prioridad**Media**
Precondiciones	Existen envíos pendientes y asignados de hoy, y al menos uno de otra fecha (cargado con`UPDATE envios SET created_at = now() - interval '2 days' WHERE ...`). Sesión de operador.
Datos de prueba	(a) Sin filtros. (b) Estado`pendiente`; estado `asignado`. (c) Fecha de hoy; `pendiente` + hoy. (d) Fecha sin envíos `2026-01-01`. (e) Id inexistente `00000000-0000-0000-0000-000000000000`.
Pasos	(1) Ir a**Envíos** y revisar las columnas con (a). (2) Aplicar los filtros (b), (c) y (d), y luego quitarlos. (3) Repetir (a)–(d) por API (`?estado=` y `&fecha=`). (4) `GET {{API}}/envios/<id de (e)>` y navegar a `/envios/<id de (e)>` en la web.
Resultado esperado	(1) Columnas: código de guía, destinatario, dirección (“calle número, localidad”), estado y fecha de creación. (2) Cada filtro muestra solo los envíos que cumplen la condición; (d) muestra un estado vacío sin error; al quitar los filtros vuelven todos. (3) La API devuelve los mismos conjuntos e incluye`fecha_creacion` en todos los ítems. (4) 404 `no_encontrado` (“el envío no existe”); la web muestra un mensaje de envío no encontrado, no una pantalla en blanco.
Responsables	Diseña: Micheloud, J. · Ejecuta: Micheloud, J. · Estado: Pendiente

HU-03 — Gestión de rutas diarias

CPF-08 — Creación de rutas y validación de obligatorios

Objetivo	Verificar que el operador crea una ruta con nombre, zona, fecha y repartidor, que queda`planificada`, y que no se puede crear sin repartidor o sin fecha.
Requisito	HU-03 · CA-01, CA-05
Clasificación	Funcional · Web + API + BD · Particiones de equivalencia · Prioridad**Alta**
Precondiciones	Sesión de operador.`TOKEN_OPE` vigente. Existe el repartidor semilla.
Datos de prueba	(a)**Válida:** `TEST Ruta Centro` · zona `Centro` · fecha de hoy · repartidora `Carla Repartidora`. (b) Sin repartidor. (c) Sin fecha. (d) Sin ninguno de los dos; por API, `repartidor_id: ""`.
Pasos	(1) Ir a**Rutas → Nueva ruta** y verificar que el selector solo lista usuarios con rol repartidor. (2) Crear (a) y verificar con `SELECT * FROM rutas WHERE nombre='TEST Ruta Centro';`. (3) Intentar guardar (b), (c) y (d) en el formulario. (4) Enviar (b) y (d) a `POST {{API}}/rutas`.
Resultado esperado	(2) 201. La ruta aparece en el listado del día como`planificada`, con zona `Centro`, la repartidora elegida y 0 envíos; en la base, los datos están completos. (3) “El repartidor es obligatorio” y/o “La fecha es obligatoria”; no se envía la petición. (4) 400 `campos_obligatorios` (“faltan el repartidor o la fecha de la ruta”). No se crea ninguna ruta con (b)–(d).
Postcondición	Ruta planificada`RUTA-HOY` (se usa en HU-04 y HU-05).
Responsables	Diseña: Díaz Aimar, F. · Ejecuta: Díaz Aimar, F. · Estado: Pendiente

CPF-09 — Listado de rutas del día y detalle de ruta

Objetivo	Verificar que se listan las rutas del día con nombre, zona, repartidor, cantidad de envíos y estado, y que el detalle muestra la ruta con sus envíos y el estado de cada uno.
Requisito	HU-03 · CA-02, CA-04
Clasificación	Funcional · Web + API · Transición de estados (una ruta por estado) · Prioridad**Media**
Precondiciones	Para hoy existen una ruta en cada estado (`planificada`, `en_curso` y `completada`; los dos últimos fijados por SQL) y una ruta de mañana. Ruta `TEST Ruta Detalle` (hoy) con 2 envíos vinculados por SQL, uno de ellos en `en_camino`.
Datos de prueba	(a) Fecha de hoy. (b)`TEST Ruta Detalle`. (c) Una ruta sin envíos. (d) Id inexistente `00000000-0000-0000-0000-000000000000`.
Pasos	(1) Ir a**Rutas** (vista del día) y revisar columnas y estados; repetir con `GET {{API}}/rutas?fecha=<hoy>`. (2) Abrir el detalle de (b). (3) Abrir el detalle de (c). (4) `GET {{API}}/rutas/<id de (d)>` y navegar a `/rutas/<id de (d)>`.
Resultado esperado	(1) Se ven las 3 rutas de hoy con su estado correcto y no la de mañana; cada fila muestra nombre, zona, repartidor y cantidad de envíos. (2) Se muestran los datos de la ruta y sus envíos con código, destinatario y estado individual (`asignado` y `en_camino`). (3) La lista de envíos está vacía; la API devuelve `"envios": []`, nunca `null`. (4) 404 `no_encontrado` (“la ruta no existe”) y un mensaje de ruta no encontrada en la web.
Responsables	Diseña: Díaz Aimar, F. · Ejecuta: Díaz Aimar, F. · Estado: Pendiente

CPF-10 — Edición de rutas según su estado

Objetivo	Verificar que una ruta`planificada` se puede editar y que las rutas `en_curso` o `completada` no admiten edición.
Requisito	HU-03 · CA-03
Clasificación	Funcional · Web + API + BD · Transición de estados (válida e inválidas) · Prioridad**Alta**
Precondiciones	`RUTA-HOY` en estado `planificada`. Rutas `TEST Ruta EnCurso` (`en_curso`) y `TEST Ruta Completa` (`completada`), fijadas por SQL porque el cambio de estado todavía no tiene pantalla.
Datos de prueba	(a)**Planificada:** nombre `TEST Ruta Centro Norte`, zona `Centro Norte`, fecha de mañana. (b) **En curso** y (c) **completada:** `{"nombre":"Editada","zona":"X","fecha":"<hoy>","repartidor_id":"33333333-3333-3333-3333-333333333333"}`
Pasos	(1) Editar`RUTA-HOY` con (a), guardar, recargar y verificar en la base. (2) Volver a ponerle la fecha de hoy. (3) Abrir el detalle de las rutas (b) y (c) en la web y verificar si se ofrece **Editar**. (4) Enviar `PUT {{API}}/rutas/<id>` con el cuerpo de prueba para (b) y (c).
Resultado esperado	(1) 200. Los cambios se ven en el detalle y en la base (`updated_at` actualizado), y el estado sigue `planificada`. (3) La web no ofrece la edición o la muestra deshabilitada. (4) 409 `ruta_no_editable` (“la ruta no se puede editar en su estado actual”) y los datos de esas rutas no cambian.
Postcondición	`RUTA-HOY` con el nombre y la zona nuevos y fecha de hoy.
Responsables	Diseña: Díaz Aimar, F. · Ejecuta: Díaz Aimar, F. · Estado: Pendiente

HU-04 — Asignación de envíos a ruta y repartidor

CPF-11 — Asignación de envíos pendientes a una ruta

Objetivo	Verificar que se pueden asignar uno o varios envíos pendientes a una ruta, que cambian automáticamente a`asignado`, que el panel solo ofrece envíos pendientes sin ruta, y que la ruta muestra la cantidad y el peso acumulado.
Requisito	HU-04 · CA-01, CA-02, CA-03, CA-06
Clasificación	Funcional · Web + API + BD · Transición de estados (pendiente → asignado) · Prioridad**Alta**
Precondiciones	`RUTA-HOY` planificada para hoy y sin envíos. `ENV-A` (3,5 kg), `ENV-B`, `ENV-C` y `ENV-D` (1 kg cada uno) pendientes. Además existen envíos en `asignado`, `en_camino` y `entregado` de otras rutas. Sesión de operador.
Datos de prueba	(a)**Individual (web):** `ENV-A`. (b) **Masiva (API), con un id repetido a propósito:** `{"envio_ids":["<ENV-B>","<ENV-C>","<ENV-C>"]}`.
Pasos	(1) Abrir el detalle de`RUTA-HOY` y revisar la lista de envíos disponibles para asignar. (2) Asignar (a) desde el panel. (3) Enviar (b) a `POST {{API}}/rutas/<RUTA-HOY>/envios`. (4) Revisar el detalle de la ruta y el listado de envíos. (5) `SELECT estado, ruta_id FROM envios WHERE id IN (<ENV-A>,<ENV-B>,<ENV-C>);`
Resultado esperado	(1) Solo aparecen envíos`pendiente` sin ruta; no aparecen los `asignado`, `en_camino` ni `entregado`. (2) 200 `{"asignados":1}`; `ENV-A` pasa a la lista de la ruta y sale de los pendientes. (3) 200 `{"asignados":2}`: los ids repetidos se descartan y se informa la cantidad realmente asignada. (4) La ruta muestra 3 envíos y 5,5 kg (`cantidad_envios: 3`, `peso_total: 5.5`), y en el listado los tres figuran como `asignado`. (5) Los tres tienen `estado='asignado'` y `ruta_id` = `RUTA-HOY`.
Postcondición	`RUTA-HOY` con 3 envíos asignados. `ENV-D` sigue pendiente.
Responsables	Diseña: Micheloud, J. · Ejecuta: Micheloud, J. · Estado: Pendiente

CPF-12 — Rechazo de asignaciones inválidas (fecha vencida y lote mixto)

Objetivo	Verificar que no se asignan envíos a rutas con fecha pasada, que la fecha de hoy sí se acepta, y que si un envío del lote no está pendiente no se asigna ninguno (todo o nada).
Requisito	HU-04 · CA-01, CA-02, CA-05 (regresión del defecto#2 del Sprint 1: asignación no atómica)
Clasificación	Funcional · API + Web + BD · Valores límite (fecha) y predicción de errores · Prioridad**Alta**
Precondiciones	Rutas planificadas con fecha de ayer, hoy (`RUTA-2`) y mañana, en UTC. `ENV-D` pendiente. `ENV-A` asignado a `RUTA-HOY` (CPF-11).
Datos de prueba	(a)**Fecha vencida:** `ENV-D` a la ruta de ayer. (b) **Fechas válidas en el límite:** `ENV-D` a la ruta de mañana (y luego desasignar) y a `RUTA-2` (hoy). (c) **Lote mixto:** crear `ENV-E` pendiente y enviar `{"envio_ids":["<ENV-E>","<ENV-A>"]}` a `RUTA-2`.
Pasos	(1) Enviar (a) por API y repetirlo desde la web. (2) Enviar (b). (3) Enviar (c). (4) Consultar el estado y el`ruta_id` de `ENV-D`, `ENV-E` y `ENV-A`.
Resultado esperado	(1) 400`fecha_vencida` ("la fecha de la ruta ya pasó"); la web muestra el error y `ENV-D` sigue pendiente. (2) 200 en ambos casos. (3) 409 `envio_no_disponible` ("uno o más envíos no están pendientes"). (4) `ENV-E` sigue `pendiente` y sin ruta; `ENV-A` sigue asignado a `RUTA-HOY`, no a `RUTA-2`.
Resultado alternativo	Como el sistema usa UTC, entre las 21:00 y las 23:59 hora argentina una ruta con la fecha local de "hoy" ya es "ayer" en UTC y se rechaza. Este comportamiento se documenta como conocido; el caso se ejecuta fuera de esa franja.
Postcondición	`ENV-D` asignado a `RUTA-2`. `ENV-E` pendiente.
Responsables	Diseña: Micheloud, J. · Ejecuta: Micheloud, J. · Estado: Pendiente

CPF-13 — Desasignación de envíos según el estado de la ruta

Objetivo	Verificar que, en una ruta planificada, un envío desasignado vuelve a `pendiente` y se desvincula, y los totales de la ruta se recalculan; y que en una ruta en curso no se permite desasignar.
Requisito	HU-04 · CA-04, CA-06
Clasificación	Funcional · Web + API + BD · Transición de estados (asignado → pendiente; transición inválida) · Prioridad **Alta**
Precondiciones	`RUTA-HOY` planificada con `ENV-A` (3,5 kg), `ENV-B` y `ENV-C` (CPF-11). Ruta `TEST Ruta EnCurso` (`en_curso`) con un envío vinculado por SQL.
Datos de prueba	(a) **Planificada:** desasignar `ENV-A` de `RUTA-HOY`. (b) **Repetir** la misma desasignación por API. (c) **En curso:** desasignar el envío de `TEST Ruta EnCurso`.
Pasos	(1) En el detalle de `RUTA-HOY`, presionar **Desasignar** sobre `ENV-A`. (2) Verificar la lista de la ruta, los totales y la lista de pendientes, y consultar la base. (3) Enviar (b): `DELETE {{API}}/rutas/<RUTA-HOY>/envios/<ENV-A>`. (4) Enviar (c) y consultar ese envío.
Resultado esperado	(1)–(2) `ENV-A` sale de la ruta y vuelve a los pendientes; la ruta pasa de 3 envíos / 5,5 kg a 2 envíos / 2 kg; en la base, `estado='pendiente'` y `ruta_id` nulo. (3) 404 `no_encontrado` ("la ruta o el envío no existen"), porque ya no está en la ruta. (4) 409 `ruta_no_editable`; el envío sigue vinculado y sin cambios.
Postcondición	`ENV-A` pendiente; `RUTA-HOY` con 2 envíos.
Responsables	Diseña: Micheloud, J. · Ejecuta: Micheloud, J. · Estado: Pendiente

HU-05 — Listado de envíos del día para repartidor (app móvil)

CPF-14 — Envíos del día del repartidor: contenido, agrupación y caso vacío

Objetivo	Verificar que la app muestra solo los envíos de las rutas de hoy del repartidor logueado, con los datos requeridos en cada tarjeta, agrupados por ruta cuando tiene más de una, y un estado vacío cuando no tiene asignaciones.
Requisito	HU-05 · CA-01, CA-02, CA-03 (regresión de los defectos #1 y #3 del Sprint 1)
Clasificación	Funcional · Móvil + API · Particiones de equivalencia (por repartidor y por cantidad de rutas) y valores límite (ayer / hoy / mañana) · Prioridad **Alta**
Precondiciones	**Repartidor 1** (`repartidor@movii.demo`): `RUTA-HOY` con `ENV-B`, `ENV-C` y `ENV-A` reasignado (`TEST Cliente Uno`, Mitre 250, San Francisco, 3,5 kg); una segunda ruta de hoy, `TEST Ruta Sur`, con 1 envío; una ruta de ayer y otra de mañana con un envío cada una. **Repartidor 2** (`repartidor2@movii.test` / `Prueba2026!`): una ruta de hoy con `ENV-R2`. **Repartidor 3** (`repartidor3@movii.test` / `Prueba2026!`): sin rutas. App abierta en Expo Go.
Datos de prueba	(a) Repartidor 1. (b) Repartidor 2. (c) Repartidor 3. (d) Con el token del repartidor 1: `GET {{API}}/repartidor/envios-del-dia?repartidor_id=<id-repartidor-2>`.
Pasos	(1) Iniciar sesión en la app con (a) y revisar "Mi ruta de hoy". (2) Comparar los datos de la tarjeta de `TEST Cliente Uno` con su detalle en la web. (3) Cerrar sesión e ingresar con (b). (4) Ingresar con (c). (5) Enviar (d) y revisar el campo `fecha` de la respuesta.
Resultado esperado	(1) Se ven solo los 4 envíos de hoy, en dos secciones con encabezado (nombre de la ruta + cantidad de envíos); no aparecen los de ayer, los de mañana ni `ENV-R2`. (2) La tarjeta muestra destinatario, "Mitre 250, San Francisco", el código de guía completo, el peso y el badge `Asignado`. (3) Solo se ve `ENV-R2`. (4) "No tenés envíos asignados para hoy." y el resumen en 0; no aparece "No se pudo cargar la ruta del día". (5) La API ignora el parámetro, devuelve los envíos del dueño del token y la `fecha` es la fecha UTC actual. Con el token del repartidor 3, la API responde 200 con `"rutas": []`.
Responsables	Diseña: Araya, U. · Ejecuta: Díaz Aimar, F. · Estado: Pendiente

CPF-15 — Filtro por estado y resumen del día

Objetivo	Verificar que los chips de filtro muestran solo los envíos del estado elegido, y que el resumen del día (total, entregados, pendientes y no entregados) es correcto, no cambia al filtrar y se actualiza cuando cambia un estado.
Requisito	HU-05 · CA-04, CA-05 (regresión de los defectos#4 y #5 del Sprint 1)
Clasificación	Funcional · Móvil + API · Particiones de equivalencia (un estado por clase) · Prioridad**Media**
Precondiciones	Partiendo de CPF-14, se agrega un envío más a`TEST Ruta Sur` y se fijan estados por SQL, de modo que el repartidor 1 tenga hoy 5 envíos: 2 `asignado`, 1 `en_camino`, 1 `entregado` y 1 `no_entregado`.
Datos de prueba	Chips: Todos / Asignados / En camino / Entregados / No entregados
Pasos	(1) Revisar la tarjeta de resumen. (2) Tocar cada chip, contar las tarjetas y revisar el resumen. (3) Pasar por SQL el envío`no_entregado` a `entregado`, hacer pull-to-refresh y elegir el chip "No entregados". (4) Volver a **Todos** y comparar el resumen con `GET {{API}}/repartidor/envios-del-dia`.
Resultado esperado	(1) Total 5 · Entregados 1 · No entregados 1 · Pendientes 3 (asignados + en camino), con un porcentaje de avance coherente. (2) Asignados 2 · En camino 1 · Entregados 1 · No entregados 1 · Todos 5; el cambio es inmediato y el resumen no cambia al filtrar. (3) "Ningún envío coincide con el filtro seleccionado." y el resumen muestra Entregados 2 · No entregados 0. (4) Los valores coinciden con`total`, `entregados`, `no_entregados`, `asignados` y `en_camino` de la API.
Postcondición	Un envío más en estado entregado.
Responsables	Diseña: Araya, U. · Ejecuta: Díaz Aimar, F. · Estado: Pendiente

Flujo integral del MVP

CPF-16 — Flujo completo: del registro del envío a la vista del repartidor

Objetivo	Validar el objetivo del Sprint 1: un operador registra un envío, lo asigna a una ruta con repartidor, y el repartidor lo ve en su app. Verifica además la interoperabilidad entre la web y la app móvil sobre la misma API.
Requisito	Objetivo del Sprint 1 · HU-02, HU-03, HU-04, HU-05
Clasificación	Funcional · E2E (Web + Móvil + API) · Basada en escenarios · Prioridad**Alta**
Precondiciones	Base restaurada con los datos semilla. Web abierta en Chrome y app abierta en un teléfono, ambas contra la misma API.
Datos de prueba	Envío: destinatario`TEST E2E`, `Belgrano` 1200, `San Francisco`, 2 kg, chico. Ruta: `TEST Ruta E2E`, zona `Este`, hoy, repartidora Carla.
Pasos	(1) Web, como operador: registrar el envío y anotar el código de guía. (2) Crear la ruta. (3) Asignarle el envío. (4) Móvil: iniciar sesión como`repartidor@movii.demo`. (5) Verificar que el envío aparece con su código, en la sección de la ruta, y que el resumen lo cuenta. (6) Web: desasignar el envío. (7) Móvil: pull-to-refresh.
Resultado esperado	Cada paso termina sin errores. En (5), los datos de la tarjeta coinciden exactamente con los registrados en (1). En (7), el envío ya no aparece y el resumen se actualiza. Los pasos 1 a 5 le llevan menos de 5 minutos a un usuario entrenado.
Postcondición	Envío pendiente; ruta planificada sin envíos.
Responsables	Diseña: Araya, U. · Ejecuta: Araya, U. · Estado: Pendiente

11.4 Casos de prueba no funcionales (RNF)

Seguridad

CPNF-01 — Protección de credenciales y autenticación con JWT

Objetivo	Verificar que las contraseñas nunca se guardan ni se devuelven en texto plano, que el tiempo de respuesta del login no permite deducir si un email existe, y que la API solo acepta tokens HS256 válidos, firmados con su secreto y no expirados.
Requisito	ISO 25010: Seguridad — Confidencialidad / Autenticidad · HU-01 · CA-01, CA-04, CA-05 (regresión del "timing oracle" y del defecto#8 del Sprint 1)
Clasificación	No funcional · BD + API · Inspección, medición y predicción de errores · Prioridad**Alta**
Precondiciones	Usuarios creados en CPF-01.`TOKEN_REP` válido. Script de firma con el `JWT_SECRET` del ambiente de pruebas. API en reposo (sin otras pruebas en curso).
Datos de prueba	(a)**Almacenamiento:** tabla `usuarios`. (b) **Tiempo de login:** 30 intentos con `operador@movii.demo` + contraseña incorrecta y 30 con `noexiste@movii.test`, alternados. (c) **Tokens inválidos:** `TOKEN_REP` con el payload cambiado a `"rol":"supervisor"` sin volver a firmar; header `"alg":"none"` sin firma; firmado con HS512; HS256 con `exp` en el pasado; HS256 firmado con otro secreto.
Pasos	(1)`SELECT email, password_hash FROM usuarios;` y revisar las respuestas de `POST /auth/register`, `POST /auth/login` y `GET /users`, y los logs de la API. (2) Ejecutar las series de (b) con `curl -w "%{time_total}"` (o k6) y calcular la mediana de cada una. (3) Enviar `GET {{API}}/users` con cada token de (c).
Resultado esperado	(1) Todos los hashes tienen formato bcrypt (`$2a$10$…`) y ninguno coincide con una contraseña de prueba; ninguna respuesta ni log incluye `password` o `password_hash`. (2) Las dos series responden 401 con un cuerpo idéntico, y la diferencia entre las medianas es menor a 50 ms. (3) Los 5 tokens responden 401 `token_invalido`, nunca datos ni 403.
Responsables	Diseña: Micheloud, J. · Ejecuta: Micheloud, J. · Estado: Pendiente

CPNF-02 — Autorización: alta de usuarios y acceso a datos según el rol

Objetivo	Verificar que solo el supervisor puede registrar usuarios (HU-01: "Yo como administrador… deseo registrar usuarios") y que un repartidor no puede consultar por la API información que su perfil no necesita (todos los envíos con los datos de los destinatarios, o las rutas de otros repartidores).
Requisito	ISO 25010: Seguridad — Confidencialidad · HU-01 ("PARA" y CA-05) · HU-05 · CA-01
Clasificación	No funcional · API · Tabla de decisión · Prioridad**Alta**
Precondiciones	`TOKEN_OPE` y `TOKEN_REP` vigentes. Existen envíos y rutas de otros repartidores.
Datos de prueba	(a)**Alta:** `{"nombre":"TEST Intruso","email":"intruso@movii.test","password":"Prueba2026!","rol":"supervisor"}` enviado sin token, con `TOKEN_REP` y con `TOKEN_OPE`. (b) **Listados con `TOKEN_REP`:** `GET /envios`, `GET /envios/<envío de otro repartidor>`, `GET /rutas` y `GET /rutas/<ruta de otro repartidor>`.
Pasos	(1) Enviar las tres variantes de (a) a`POST {{API}}/auth/register` y verificar si existe `intruso@movii.test`. (2) Enviar las cuatro peticiones de (b).
Resultado esperado	(1) Sin token: 401; con`TOKEN_REP` o `TOKEN_OPE`: 403. No se crea el usuario. (2) 403 `rol_no_autorizado`, o solo información de las rutas propias del repartidor.
Resultado alternativo	Si (1) sin token responde 201, o si (2) devuelve datos de todos los clientes, se registra un defecto de severidad**Crítica**: en el primer caso cualquier persona podría crearse una cuenta de supervisor; en el segundo, se exponen nombres y direcciones de destinatarios a cualquier repartidor.

Postcondición	Sin cambios (si se creó el usuario intruso, borrarlo).
Responsables	Diseña: Micheloud, J. · Ejecuta: Micheloud, J. · Estado: Pendiente

CPNF-03 — Integridad de las entradas: inyección SQL y política CORS

Objetivo	Verificar que los parámetros no permiten alterar las consultas SQL, y que la API solo acepta peticiones de navegador desde los orígenes configurados.
Requisito	ISO 25010: Seguridad — Integridad · Configuración`CORS_ORIGINS` (Sprint 0, 7.2)
Clasificación	No funcional · API · Predicción de errores · Prioridad**Media**
Precondiciones	`TOKEN_OPE` vigente. `CORS_ORIGINS=http://localhost:5173`. Contar previamente los envíos y las rutas.
Datos de prueba	(a)**Inyección SQL**, con los caracteres codificados en la URL: `GET /envios?estado=' OR '1'='1`; `GET /envios?fecha=2026-01-01'; DROP TABLE envios;--`; `GET /rutas/1' OR '1'='1`; `POST /envios` con el destinatario `TEST Robert'); DELETE FROM envios;--`. (b) **CORS:** preflight `OPTIONS /envios` con `Origin: http://localhost:5173` y con `Origin: http://evil.example`.
Pasos	(1) Enviar las peticiones de (a) y volver a contar envíos y rutas. (2) Enviar los dos preflight de (b) con`Access-Control-Request-Method: POST`.
Resultado esperado	(1) Los`GET` responden 200 con una lista vacía o 4xx, nunca todos los registros ni un 500 con el detalle del error SQL. El `POST` registra el nombre literal. Las cantidades no cambian, salvo por ese envío. (2) Con el origen permitido, la respuesta incluye `Access-Control-Allow-Origin: http://localhost:5173`; con `evil.example` no incluye ese header, por lo que el navegador bloquea la petición.
Postcondición	Un envío con nombre literal (se limpia por su prefijo`TEST`).
Responsables	Diseña: Micheloud, J. · Ejecuta: Micheloud, J. · Estado: Pendiente

Eficiencia de desempeño y fiabilidad

CPNF-04 — Tiempo de respuesta y concurrencia de la API bajo carga

Objetivo	Verificar que la API responde dentro de tiempos aceptables con una concurrencia similar a la de una empresa de reparto mediana al inicio de la jornada, y que las altas simultáneas de envíos generan códigos de guía únicos.
Requisito	ISO 25010: Eficiencia de desempeño — Comportamiento temporal · Fiabilidad — Madurez · Valor de negocio “eficiencia operativa” (Sprint 0, 3.1) · HU-02 · CA-02
Clasificación	No funcional · API · Prueba de carga y de concurrencia (k6) · Prioridad**Alta**
Precondiciones	Base de pruebas con al menos 200 envíos y 10 rutas del día (cargados con un script). API en la PC de pruebas, con conexión estable.
Datos de prueba	(a)**Lecturas:** 20 usuarios virtuales durante 2 minutos; 50 % `GET /repartidor/envios-del-dia`, 30 % `GET /envios`, 20 % `GET /rutas?fecha=<hoy>`; login previo en el `setup`. (b) **Login:** 10 logins concurrentes. (c) **Altas concurrentes:** 10 usuarios virtuales que envían 5 `POST /envios` cada uno, en simultáneo (50 altas).
Pasos	(1) Ejecutar (a) y exportar el resumen de k6 (p50, p95, tasa de error). (2) Ejecutar (b). (3) Ejecutar (c), contar las respuestas por código HTTP y ejecutar`SELECT codigo_guia, count(*) FROM envios GROUP BY 1 HAVING count(*)>1;`.
Resultado esperado	(1) p95 < 1 s, tasa de errores < 1 % y ninguna respuesta 5xx. (2) p95 < 1,5 s (incluye el costo intencional de bcrypt). (3) Al menos el 98 % de las altas responde 201; las que fallan lo hacen con un error controlado y sin dejar datos parciales; no hay códigos duplicados.
Postcondición	Datos de carga eliminados con el script de limpieza.

Responsables	Diseña: Díaz Aimar, F. · Ejecuta: Díaz Aimar, F. · Estado: Pendiente

CPNF-05 — Tiempo de carga de las pantallas principales

Objetivo	Verificar que las pantallas de uso diario cargan rápido en las condiciones reales de uso.
Requisito	ISO 25010: Eficiencia de desempeño — Comportamiento temporal · Decisión de diseño “reducir el tiempo de operación” (Sprint 0, 8.4)
Clasificación	No funcional · Web + Móvil · Medición (Lighthouse, cronómetro) · Prioridad**Media**
Precondiciones	Datos de CPNF-04 cargados. Teléfonos en la red WiFi de pruebas.
Pasos	(1) Lighthouse (modo navegación, móvil) sobre`/envios`, con sesión iniciada. (2) En la app, medir 5 veces el tiempo entre tocar **Ingresar** y ver la lista de “Mi ruta de hoy”. (3) Medir 5 veces el pull-to-refresh.
Resultado esperado	Web: puntaje de Performance ≥ 80 y LCP < 2,5 s. Móvil: login → lista < 3 s y refresh < 2 s (promedio de 5 mediciones), en los dos dispositivos.
Responsables	Diseña: Díaz Aimar, F. · Ejecuta: Díaz Aimar, F. · Estado: Pendiente

Compatibilidad y portabilidad

CPNF-06 — Funcionamiento en los navegadores y dispositivos soportados

Objetivo	Verificar que el panel web funciona en los navegadores soportados y que la app del repartidor, generada desde un único proyecto, funciona igual en Android y en iOS.
Requisito	ISO 25010: Compatibilidad — Coexistencia / Interoperabilidad · Portabilidad — Adaptabilidad · Stack (Sprint 0, 6.2: “genera versiones Android e iOS desde un mismo proyecto”)
Clasificación	No funcional · Web + Móvil · Matriz de compatibilidad · Prioridad**Alta**
Precondiciones	Chrome, Firefox y Edge en su última versión estable. iPhone y Android con Expo Go (SDK 57) en la red de pruebas.
Datos de prueba	(a)**Web**, en los 3 navegadores: CPF-02 (a), CPF-05 (a), CPF-06 (a), CPF-08 (a), CPF-11 y CPF-13 (a). (b) **Móvil**, en los 2 dispositivos: login fallido (CPF-02 (b)), CPF-14 y CPF-15.
Pasos	(1) Ejecutar (a) en cada navegador, registrar su versión exacta y revisar la consola en busca de errores. (2) Ejecutar (b) en cada dispositivo, en orientación vertical y con el tamaño de letra del sistema aumentado; registrar el modelo y la versión del SO.
Resultado esperado	(1) Todos los casos pasan en los 3 navegadores, sin errores de consola, sin roturas de layout y con la misma validación de formularios. (2) Todos los casos pasan en ambos dispositivos; el teclado no tapa los campos del login; con la letra aumentada, los textos no se superponen ni se cortan sin indicación.
Postcondición	Datos de prueba creados (se limpian).
Responsables	Diseña: Araya, U. y Díaz Aimar, F. · Ejecuta: Araya, U. (web) y Díaz Aimar, F. (móvil) · Estado: Pendiente

CPNF-07 — Instalabilidad del ambiente según el instructivo

Objetivo	Verificar que un integrante puede replicar el ambiente completo en una máquina limpia siguiendo solo el instructivo del Sprint 0.
Requisito	ISO 25010: Portabilidad — Instalabilidad · Sprint 0, 7.2
Clasificación	No funcional · Instalación · Basada en checklist · Prioridad**Baja**
Precondiciones	Máquina (o usuario de Windows nuevo) sin el repositorio clonado, con los prerrequisitos de la sección 7.2.
Datos de prueba	Instructivo de instalación del Sprint 0
Pasos	(1) Seguir el instructivo paso a paso sin ayuda externa. (2) Cronometrar. (3) Anotar cada paso ambiguo o que falla. (4) Verificar con`GET /health` y con un login desde la app.

Resultado esperado	El ambiente queda operativo (API, migraciones y app en el teléfono) en menos de 60 minutos y sin pasos no documentados. Las desviaciones se registran como defectos de documentación (severidad Baja).
Postcondición	Ambiente adicional disponible.
Responsables	Diseña: Díaz Aimar, F. · Ejecuta: Araya, U. · Estado: Pendiente

Usabilidad

CPNF-08 — Diseño mobile-first, consistencia visual y accesibilidad básica

Objetivo	Verificar que la vista del repartidor se adapta a 390×844 px, que se cumplen las decisiones de diseño del prototipo (colores semánticos por estado, flujos críticos de 3 pasos como máximo) y criterios básicos de accesibilidad.
Requisito	ISO 25010: Usabilidad — Operabilidad / Estética de la interfaz / Accesibilidad · HU-05 · CA-06 · Sprint 0, 8.4
Clasificación	No funcional · Móvil + Web · Inspección contra checklist + Lighthouse · Prioridad**Alta**
Precondiciones	iPhone de 390×844 pt (o emulación 390×844 en Chrome DevTools para el panel web). Repartidor con 5 envíos en 2 rutas, en todos los estados.
Datos de prueba	Checklist mobile-first: sin scroll horizontal; resumen, chips y tarjetas visibles sin solaparse; zonas táctiles de al menos 44×44 pt; textos de al menos 13 pt. Colores esperados: verde = entregado, azul = en camino, amarillo = pendiente/asignado, rojo = no entregado.
Pasos	(1) Abrir “Mi ruta de hoy”, aplicar el checklist y hacer scroll hasta el final. (2) Repetir el checklist con el panel web en 390×844. (3) Comparar los colores de los badges en la web y en la app. (4) Contar las interacciones para asignar un envío desde el detalle de una ruta, y las acciones de cada tarjeta de la app. (5) Lighthouse (Accessibility) sobre`/login` y `/envios/nuevo`; completar el login usando solo el teclado; recorrer el login de la app con VoiceOver.
Resultado esperado	(1) La app cumple el 100 % del checklist. (2) En el panel web, envíos y rutas se usan sin scroll horizontal de la página (las tablas pueden tener scroll propio). (3) Los colores coinciden en las dos interfaces y en todas las pantallas. (4) Asignar requiere 3 interacciones o menos y ninguna tarjeta tiene más de 3 acciones. (5) Accessibility ≥ 90; todos los campos tienen etiqueta y el foco es visible; VoiceOver anuncia “Mantener sesión iniciada” como casilla y “Recuperar contraseña” como enlace.
Responsables	Diseña: Araya, U. · Ejecuta: Díaz Aimar, F. (móvil) y Araya, U. (web) · Estado: Pendiente

CPNF-09 — Prueba de usabilidad con usuarios (SUS)

Objetivo	Validar con usuarios representativos que las tareas principales se completan sin ayuda y con un nivel de satisfacción aceptable.
Requisito	ISO 25010: Usabilidad — Reconocimiento de adecuación / Capacidad de aprendizaje / Protección contra errores del usuario
Clasificación	No funcional · Web + Móvil · Prueba con usuarios + cuestionario SUS (System Usability Scale) · Prioridad**Media**
Precondiciones	5 participantes externos al proyecto. Ambiente con datos semilla. Guion de tareas impreso.
Datos de prueba	T1 (web, operador): registrar un envío con los datos de una ficha impresa · T2 (web): crear una ruta para hoy y asignarle el envío · T3 (móvil, repartidor): encontrar el envío de T1 y decir su peso · T4 (móvil): filtrar los envíos no entregados
Pasos	(1) Explicar el objetivo sin dar instrucciones de uso. (2) Pedir a cada participante que realice T1–T4 pensando en voz alta. (3) Registrar el tiempo, los errores y si necesitó ayuda. (4) Al final, completar el cuestionario SUS de 10 preguntas.

Resultado esperado	Al menos 4 de los 5 participantes completan cada tarea sin ayuda. Puntaje SUS promedio ≥ 68 (por encima del promedio de la industria). Cada problema observado en 2 o más participantes se registra como mejora o defecto.
Postcondición	Informe de usabilidad adjunto al informe de estado del sprint.
Responsables	Diseña: Araya, U. · Ejecuta: Araya, U. · Estado: Pendiente

Fiabilidad

CPNF-10 — Tolerancia a fallos de conectividad

Objetivo	Verificar que, si la API o la red no están disponibles, las interfaces informan el problema de forma comprensible y se recuperan sin reiniciar la app.
Requisito	ISO 25010: Fiabilidad — Tolerancia a fallos / Capacidad de recuperación · Operación del repartidor en la calle, con conectividad variable
Clasificación	No funcional · Móvil + Web · Inyección de fallos (detener la API, modo avión) · Prioridad**Alta**
Precondiciones	Sesión iniciada en la app y en la web.
Pasos	(1) Detener la API. (2) En la app, hacer pull-to-refresh. (3) En la web, intentar registrar un envío. (4) Volver a levantar la API. (5) Repetir el refresh y el registro. (6) Repetir (2) con el teléfono en modo avión y luego desactivarlo.
Resultado esperado	(2) La app muestra “No se pudo cargar la ruta del día” (o un mensaje equivalente), no se cierra y conserva la última lista o un estado de error con opción de reintentar. (3) La web muestra “No se pudo registrar el envío”, conserva los datos del formulario y no queda en blanco. (5) Ambas operaciones funcionan sin reiniciar ni volver a iniciar sesión. (6) Igual que (2) y (5).
Postcondición	API operativa.
Responsables	Diseña: Díaz Aimar, F. · Ejecuta: Díaz Aimar, F. · Estado: Pendiente

Mantenibilidad

CPNF-11 — Cobertura de pruebas unitarias, estándares de codificación y CI

Objetivo	Verificar los criterios de calidad interna del DoD: cobertura mínima del 70 % en la lógica de negocio, código conforme a los estándares y pipelines en verde.
Requisito	ISO 25010: Mantenibilidad — Capacidad de ser probado / Modificabilidad · DoD puntos 2, 4 y 5
Clasificación	No funcional · Código fuente · Caja blanca (cobertura de sentencias) + análisis estático · Prioridad**Media**
Precondiciones	Repositorio actualizado en la rama a presentar. Base de pruebas disponible.
Pasos	(1) En`services/api`: `go test ./... -p 1 -coverprofile=cover.out` y `go tool cover -func=cover.out`. (2) `gofmt -l .` y `go vet ./...`. (3) En `apps/web` y `apps/mobile`: lint y tests. (4) Verificar el estado de `api-ci.yml`, `web-ci.yml` y `mobile-ci.yml` en GitHub Actions. (5) Correr dos veces seguidas la suite de la API y verificar que no deja filas huérfanas (regresión del defecto #7 del Sprint 1).
Resultado esperado	Cobertura ≥ 70 % en el paquete`internal/services`. `gofmt -l` no lista archivos y `go vet` no reporta problemas. ESLint sin errores. Todas las suites pasan (en el Sprint 1: 82 tests de la API y 19 de la app móvil, más los de la web). Las 3 pipelines están en verde. La cantidad de filas en `rutas` y `envios` es la misma antes y después de las dos corridas.
Postcondición	Reporte de cobertura adjunto como evidencia.
Responsables	Diseña: Micheloud, J. · Ejecuta: Micheloud, J. · Estado: Pendiente

11.5 Resumen de casos de prueba

Los casos `CPF` son funcionales y los `CPNF`, no funcionales.

ID	Título	Requisito	Nivel	Prioridad	Asignado a
CPF-01	Alta de usuarios y validación de sus datos	HU-01 CA-01, CA-02	Web + API + BD	Alta	Araya
CPF-02	Inicio de sesión válido e inválido	HU-01 CA-03, CA-04	API + Web + Móvil	Alta	Araya
CPF-03	Control de acceso por token y por rol	HU-01 CA-05	API + Web	Alta	Araya
CPF-04	Listado de usuarios	HU-01 CA-06	Web + API	Media	Araya
CPF-05	Registro de envío y código de guía	HU-02 CA-01, CA-02, CA-03	Web + API + BD	Alta	Micheloud
CPF-06	Validación de obligatorios y del peso	HU-02 CA-04, CA-06	Web + API	Alta	Micheloud
CPF-07	Listado de envíos con filtros y detalle	HU-02 CA-05	Web + API	Media	Micheloud
CPF-08	Creación de rutas y obligatorios	HU-03 CA-01, CA-05	Web + API + BD	Alta	Díaz Aimar
CPF-09	Listado del día y detalle de ruta	HU-03 CA-02, CA-04	Web + API	Media	Díaz Aimar
CPF-10	Edición de rutas según su estado	HU-03 CA-03	Web + API + BD	Alta	Díaz Aimar
CPF-11	Asignación de envíos pendientes	HU-04 CA-01, CA-02, CA-03, CA-06	Web + API + BD	Alta	Micheloud
CPF-12	Rechazo de asignaciones inválidas	HU-04 CA-01, CA-02, CA-05	API + Web + BD	Alta	Micheloud
CPF-13	Desasignación según el estado de la ruta	HU-04 CA-04, CA-06	Web + API + BD	Alta	Micheloud
CPF-14	Envíos del día: contenido, agrupación y caso vacío	HU-05 CA-01, CA-02, CA-03	Móvil + API	Alta	Díaz Aimar
CPF-15	Filtro por estado y resumen del día	HU-05 CA-04, CA-05	Móvil + API	Media	Díaz Aimar
CPF-16	Flujo integral del MVP	Objetivo del Sprint 1	E2E	Alta	Araya
CPNF-01	Protección de credenciales y JWT	Seguridad	BD + API	Alta	Micheloud
CPNF-02	Autorización por rol (alta y datos)	Seguridad	API	Alta	Micheloud
CPNF-03	Inyección SQL y CORS	Seguridad	API	Media	Micheloud
CPNF-04	Tiempo de respuesta y concurrencia	Eficiencia de desempeño / Fiabilidad	API	Alta	Díaz Aimar
CPNF-05	Tiempo de carga de pantallas	Eficiencia de desempeño	Web + Móvil	Media	Díaz Aimar
CPNF-06	Navegadores y dispositivos	Compatibilidad / Portabilidad	Web + Móvil	Alta	Araya / Díaz Aimar
CPNF-07	Instalabilidad del ambiente	Portabilidad	Instalación	Baja	Araya

ID	Título	Requisito	Nivel	Prioridad	Asignado a
CPNF-08	Mobile-first, consistencia y accesibilidad	Usabilidad	Móvil + Web	Alta	Díaz Aimar / Araya
CPNF-09	Prueba con usuarios (SUS)	Usabilidad	Web + Móvil	Media	Araya
CPNF-10	Tolerancia a fallos de conectividad	Fiabilidad	Móvil + Web	Alta	Díaz Aimar
CPNF-11	Cobertura, estándares y CI	Mantenibilidad	Código	Media	Micheloud

Prioridad	Funcionales	No funcionales	Total
Alta	12	6	18
Media	4	4	8
Baja	0	1	1
Total	**16**	**11**	**27**